In [1]:
```
# Import Libraries

import numpy as np
import pandas as pd

# Read the file

df = pd.read_csv("Uncleaned.csv")
```

In [2]:
```
df.head()
```

Out[2]:

	Employee ID	Employee Name	Employee Position	Salary	Age	Gender	Department	Hire Date
0	178	Christopher Reilly	employee	99280.0	62.0	Male	Accounts	2020-07-28
1	2	Jennifer Newton	employee	66717.0	NaN	Male	NaN	2018-11-04
2	42	Dakota Khan	Administrator	87759.0	56.0	Female	Accounts	2023-03-30
3	177	Nancy Long	Techies	75973.0	59.0	Male	Manegement	2018-06-27
4	55	Jeffrey Miller	Techies	67856.0	45.0	Male	Administration	2022-03-19

In [3]:
```
df.shape
```

Out[3]:
```
(208, 8)
```

In [4]:
```
print("Number of rows are:", df.shape[0])
print("Number of columns are:", df.shape[1])
```

```
Number of rows are: 208
Number of columns are: 8
```

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 208 entries, 0 to 207
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Employee ID        208 non-null    int64  
 1   Employee Name      208 non-null    str    
 2   Employee Position  208 non-null    str    
 3   Salary             205 non-null    str    
 4   Age                206 non-null    float64
 5   Gender             207 non-null    str    
 6   Department         205 non-null    str    
 7   Hire Date          207 non-null    str    
dtypes: float64(1), int64(1), str(6)
memory usage: 24.2 KB
```

In [6]:
```
missing_values = df.isnull().sum()

print(missing_values)
```

```
Employee ID          0
Employee Name        0
Employee Position    0
Salary               3
Age                  2
Gender               1
Department           3
Hire Date            1
dtype: int64
```

In [7]:
```
df["Salary"] = df["Salary"].replace("Sixty Thousand",60000)
```

In [8]:
```
df["Salary"] = pd.to_numeric(df["Salary"])
```

In [9]:
```
# Replace all the outside salaries (<50000 and >100000) with nan.

df.loc[(df["Salary"]<50000) | (df["Salary"]>100000), "Salary"] = np.nan
```

In [10]:
```
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
```

In [11]:
```
# Replace all the outlier age (>65 and <20) into nan

df.loc[(df["Age"]<20) | (df["Age"]>65), "Age"] = np.nan
```

In [12]:
```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

In [13]:
```
# Converting Age column from float to int

df["Age"] = df["Age"].astype(int)
```

In [14]:
```
# Converting Salary column from float to int

df["Salary"] = df["Salary"].astype(int)
```

In [15]:
```
df["Gender"].value_counts()
```

Out[15]:
```
Gender
Female           107
Male              98
Unknown            1
Not Disclosed      1
Name: count, dtype: int64
```

In [16]:
```
df["Gender"] = df["Gender"].replace("Unknown","Others")
df["Gender"] = df["Gender"].replace("Not Disclosed","Others")
```

In [17]:
```
df["Gender"] = df["Gender"].fillna("Unknown")
```

In [18]:
```
df["Department"].value_counts()
```

Out[18]:
```
Department
Manegement         53
Sales              47
Administration     41
Accounts           34
Human_Resources    29
Sales&Marketing     1
Name: count, dtype: int64
```

In [19]:
```
df["Department"] = df["Department"].replace("Sales&Marketing","Sales")
df["Department"] = df["Department"].replace("Manegement","Management")
```

In [20]:
```
df["Department"] = df["Department"].fillna(df["Department"].mode()[0])
```

In [21]:
```
df["Hire Date"] = pd.to_datetime(df["Hire Date"],errors = "coerce").fillna("Invalid Date")
```

In [22]:
```
# Check if any duplicate Employee ID exist or not

total_duplicates = df.duplicated().sum()
print(total_duplicates)
```

```
5
```

In [23]:
```
duplicate_rows_first = df[df["Employee ID"].duplicated(keep="first")]
print(duplicate_rows_first)
```

```
     Employee ID       Employee Name Employee Position  Salary  Age  Gender  \
67           167       Michael Brown     Administrator   86244   47    Male
102          116        Mark Pollard           Manager   62638   31    Male
104           70      Connor Holland           Manager   84196   65    Male
139            2     Jennifer Newton          employee   66717   64    Male
150            3    Diane Montgomery           Techies   80701   31    Male
162           79           Kelly Kim     Administrator   83012   62  Female
175          173  Elizabeth Thornton           Manager   99903   45    Male
204          196      Ryan Daugherty          employee   75216   59  Female

          Department            Hire Date
67        Management  2021-03-28 00:00:00
102            Sales  2023-04-15 00:00:00
104       Management  2020-12-07 00:00:00
139       Management  2018-11-04 00:00:00
150            Sales  2022-11-07 00:00:00
162   Administration  2017-12-03 00:00:00
175  Human_Resources  2018-12-08 00:00:00
204       Management  2020-03-20 00:00:00
```

In [24]:
```
df["Employee Position"].value_counts()
```

Out[24]:
```
Employee Position
employee         56
Techies          52
Manager          50
Administrator    49
Tech@ies          1
Name: count, dtype: int64
```

In [25]:
```
df["Employee Position"] = df["Employee Position"].replace("employee","Employee")
df["Employee Position"] = df["Employee Position"].replace("Tech@ies","Techies")
```

In [26]:
```
# Drop Duplicate Rows

df = df.drop_duplicates(subset="Employee ID", keep = "first")
```

In [30]:
```
df.head()
```

Out[30]:

	Employee ID	Employee Name	Employee Position	Salary	Age	Gender	Department	Hire Date
0	178	Christopher Reilly	Employee	99280	62	Male	Accounts	2020-07-28 00:00:00
1	2	Jennifer Newton	Employee	66717	47	Male	Management	2018-11-04 00:00:00
2	42	Dakota Khan	Administrator	87759	56	Female	Accounts	2023-03-30 00:00:00
3	177	Nancy Long	Techies	75973	59	Male	Management	2018-06-27 00:00:00
4	55	Jeffrey Miller	Techies	67856	45	Male	Administration	2022-03-19 00:00:00

In [28]:
```
df.to_excel("Cleaned_Employee_Data.xlsx", index=False)
```

In [31]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [39]:
```
# Visualise the count of male, female & other categories in the company

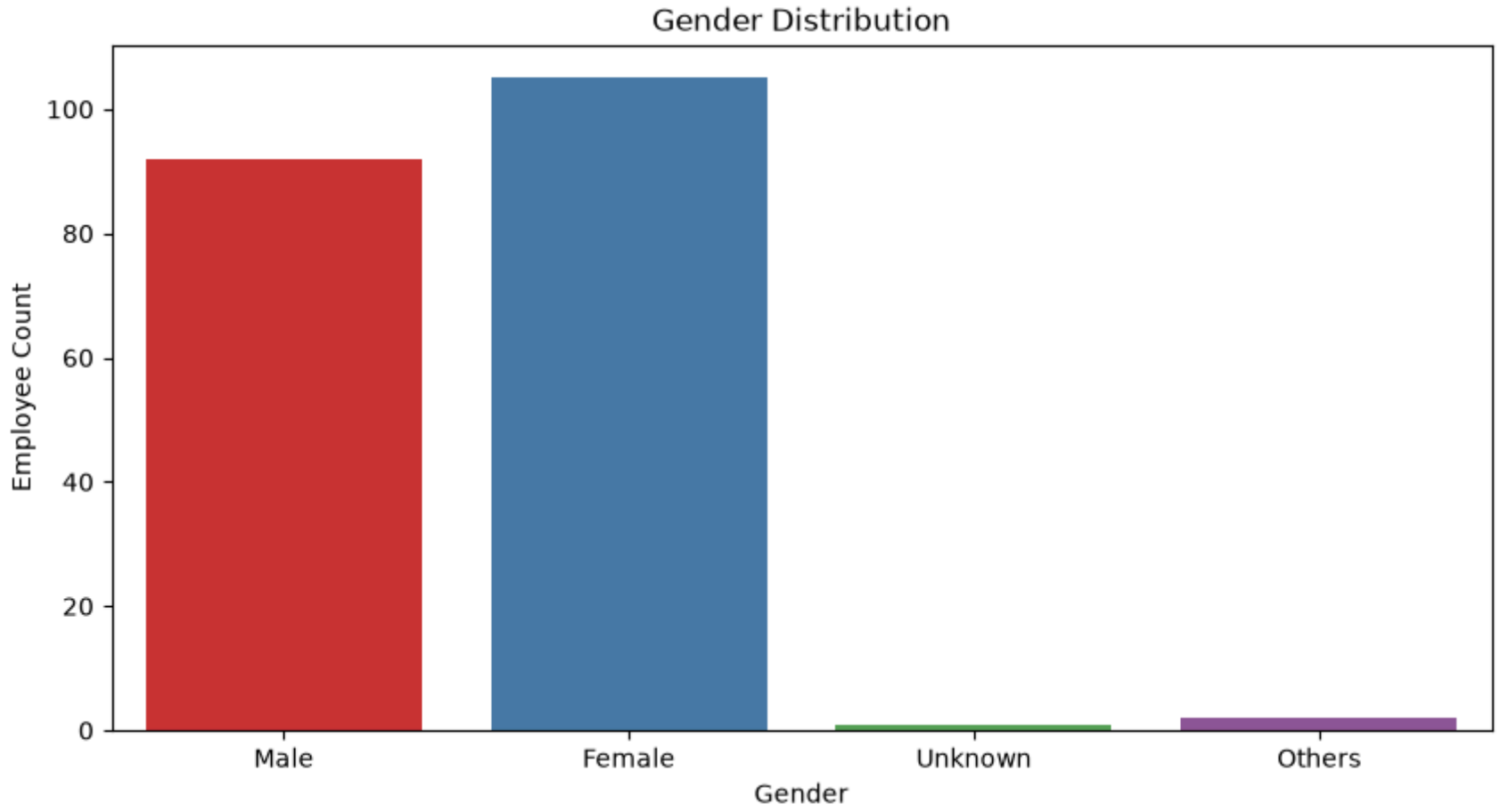
plt.figure(figsize=(10,5))
sns.countplot(x="Gender", data=df,palette="Set1")
plt.title("Gender Distribution")
plt.xlabel("Gender")
```

```
plt.ylabel("Employee Count")
plt.show()
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_1264\1178777829.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `h
ue` and set `legend=False` for the same effect.

  sns.countplot(x="Gender", data=df,palette="Set1")
```



In [45]:
```
# Visualise the Salary Distribution of the employees

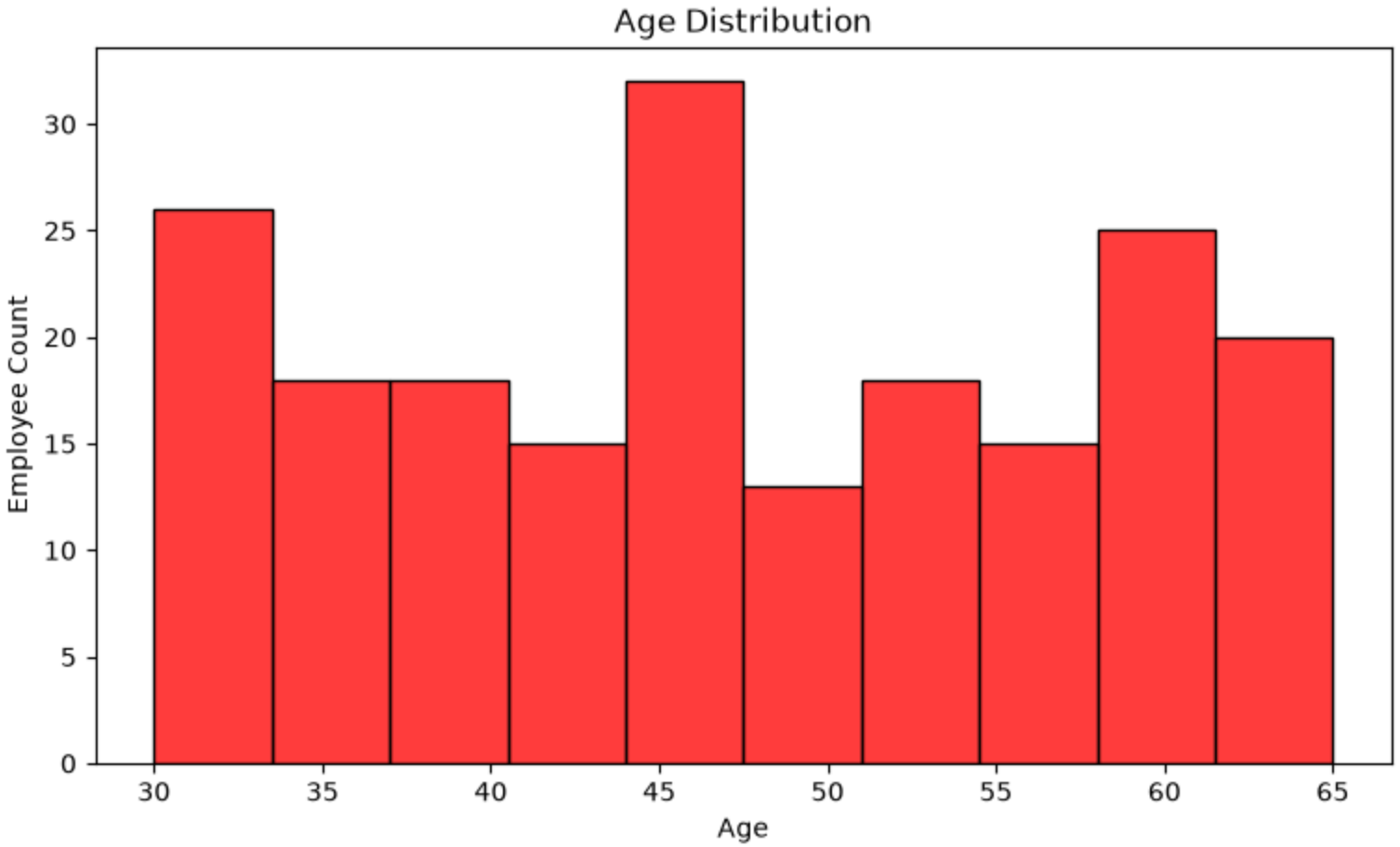
plt.figure(figsize=(9,5))
sns.histplot(x="Salary", data=df,bins=15,color="Red")
```

```
plt.title("Salary Distribution")
plt.ylabel("Employee Count")
plt.show()
```



In [48]:
```
# Visualise the Age Distribution of the employees

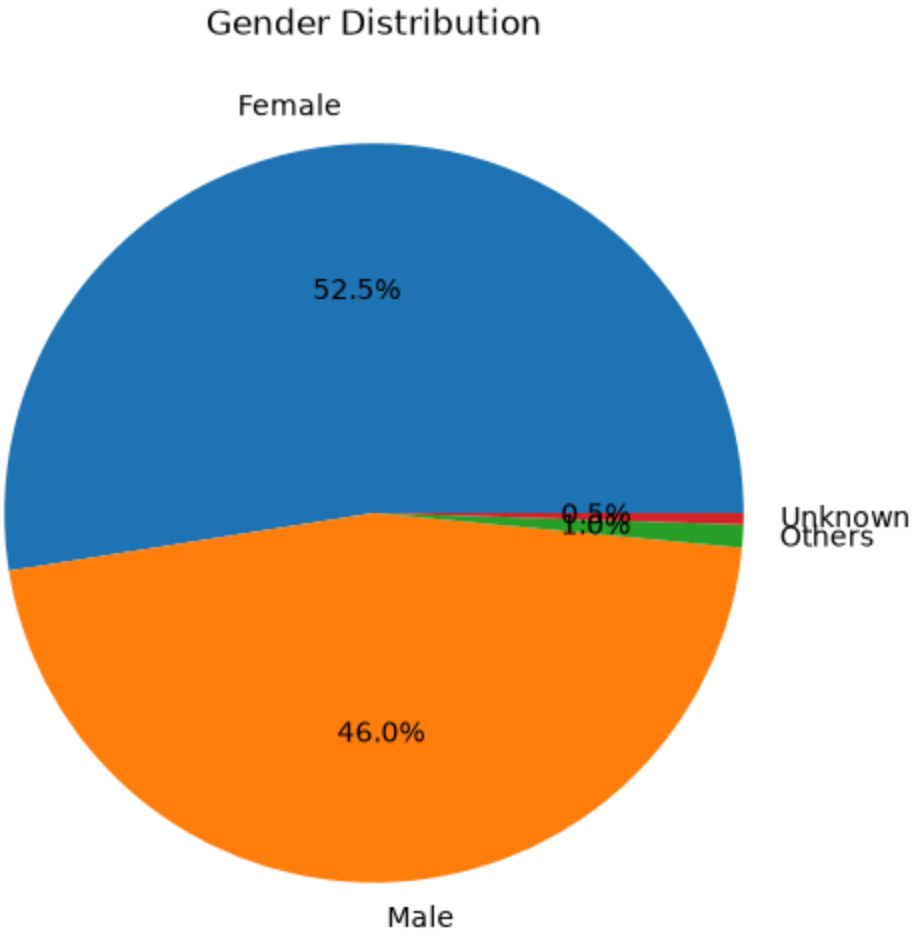
plt.figure(figsize=(9,5))
sns.histplot(x="Age", data=df,bins=10,color="Red")
plt.title("Age Distribution")
plt.ylabel("Employee Count")
plt.show()
```



In [54]:
```
# Visualise a pie chart to show the Gender

data = df["Gender"].value_counts()

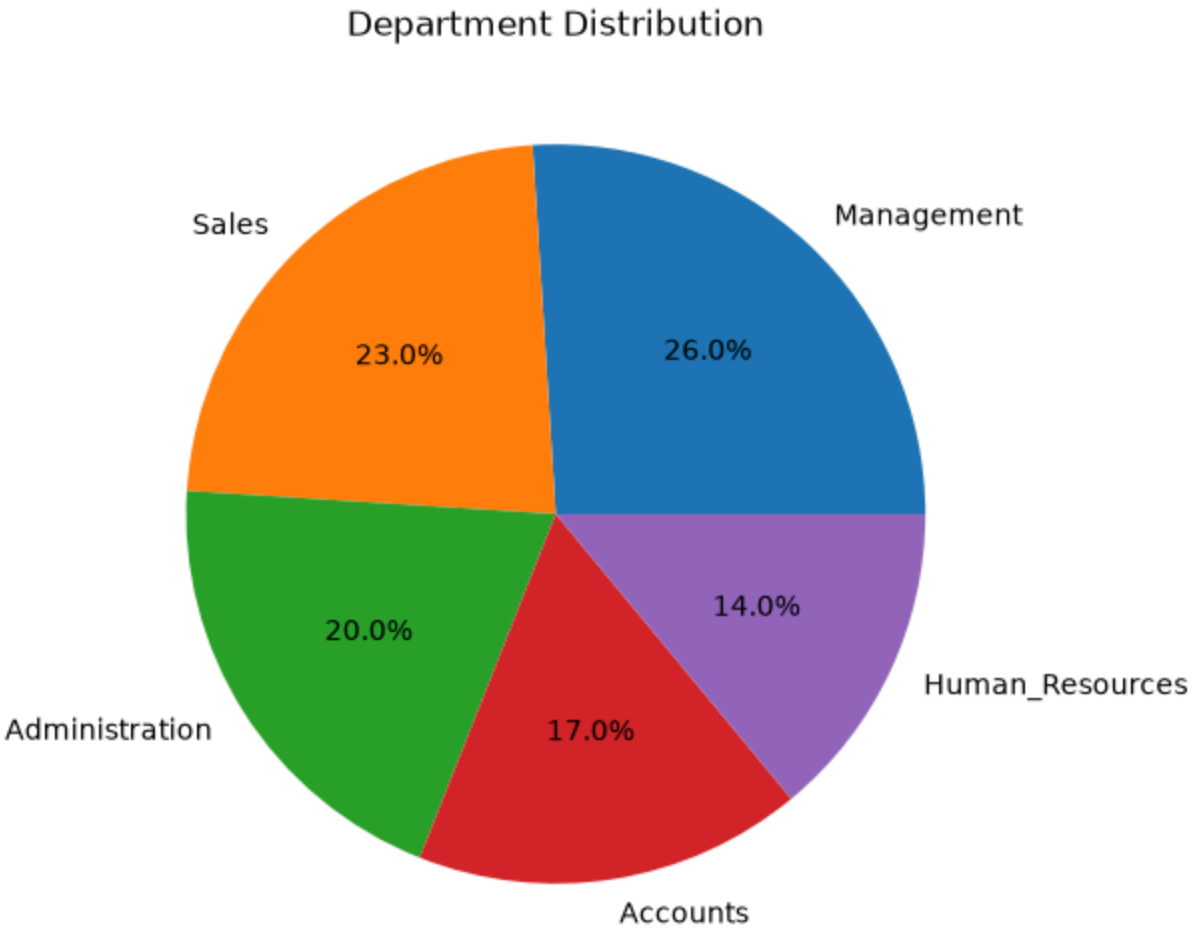
plt.figure(figsize=(9,6))
plt.pie(data,labels=data.index, autopct="%.1f%%")
plt.title("Gender Distribution")
plt.show()
```



In [57]:
```
# Create a pie chart to show what % of employees are in each department

data = df["Department"].value_counts()

plt.figure(figsize=(9,6))
plt.pie(data,labels=data.index, autopct="%.1f%%")
plt.title("Department Distribution")
plt.show()
```



In [59]:
```
# Show the hiring trend in the company over the years

df["Hire Date"] = pd.to_datetime(df["Hire Date"],errors="coerce")
df["Hire Year"] = df["Hire Date"].dt.year
```

In [61]:
```
df.head()
```

Out[61]:

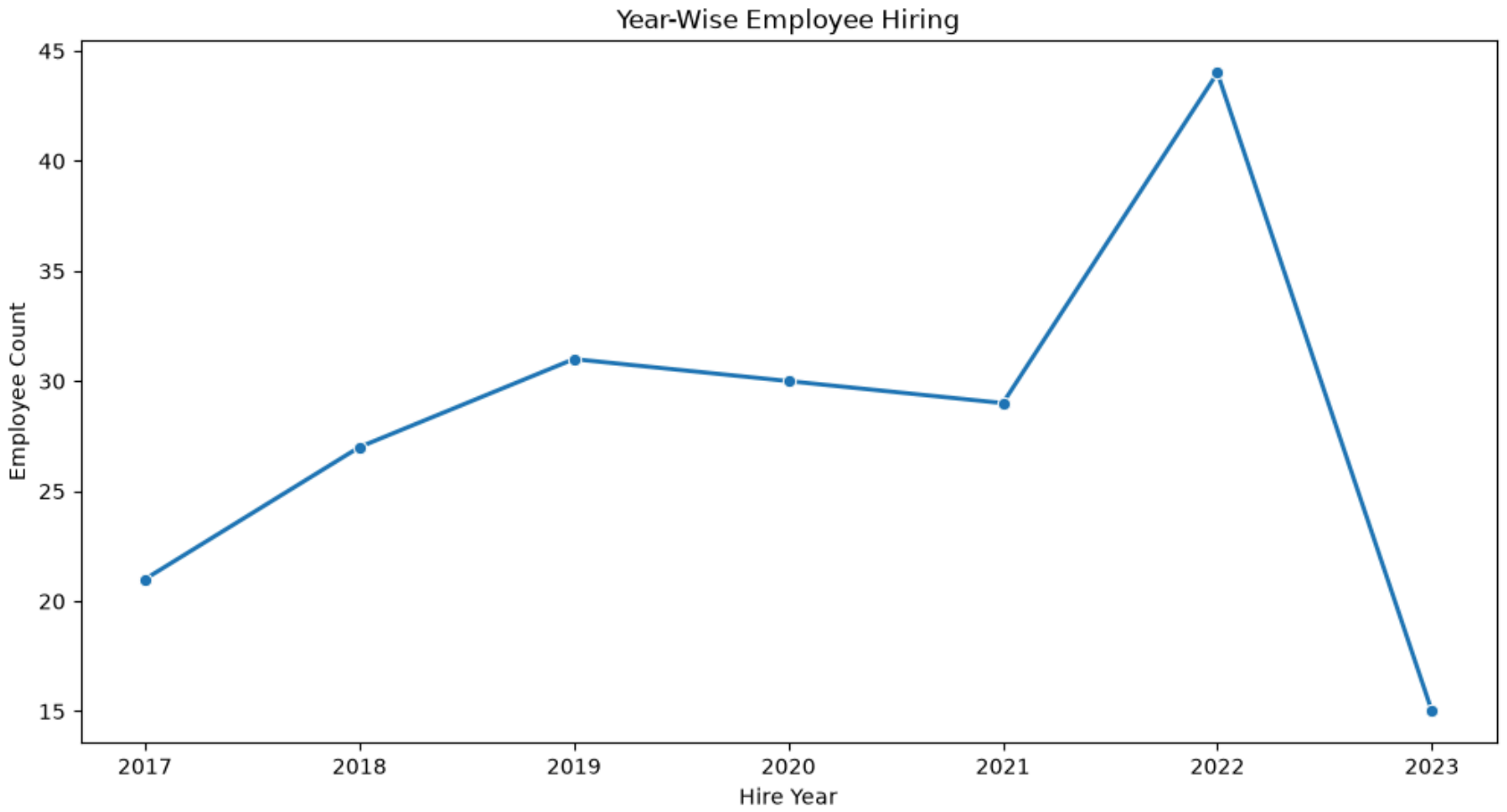
	Employee ID	Employee Name	Employee Position	Salary	Age	Gender	Department	Hire Date	Hire Year
0	178	Christopher Reilly	Employee	99280	62	Male	Accounts	2020-07-28	2020.0
1	2	Jennifer Newton	Employee	66717	47	Male	Management	2018-11-04	2018.0
2	42	Dakota Khan	Administrator	87759	56	Female	Accounts	2023-03-30	2023.0
3	177	Nancy Long	Techies	75973	59	Male	Management	2018-06-27	2018.0
4	55	Jeffrey Miller	Techies	67856	45	Male	Administration	2022-03-19	2022.0

In [69]:
```
# Summarize employee hired in each year

yearly_hiring = df["Hire Year"].value_counts().sort_index().reset_index()

yearly_hiring.columns = ["Hire Year", "Employee Count"]
```

In [71]:
```
plt.figure(figsize=(12,6))
sns.lineplot(x="Hire Year", y="Employee Count", data=yearly_hiring, marker="o", linewidth=2)
plt.title("Year-Wise Employee Hiring")
plt.show()
```


Year-Wise Employee Hiring
Employee Count
45
40
35
30
25
20
15
2017
2018
2019
2020
2021
2022
2023
Hire Year

In [ ]: